Programa:	HelpandManual 4.0
Protección:	Time-Trial 14 días
Objetivos:	Evitar la limitación temporal
URL descarga:	http://www.helpandmanual.com
Dificultad:	Casi agota la paciencia del cracker ;-)
Herramientas:	OllyDbg v1.10 + PEiD 0.94
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 02-01-06

Hola a todo CracksLatinos..!!

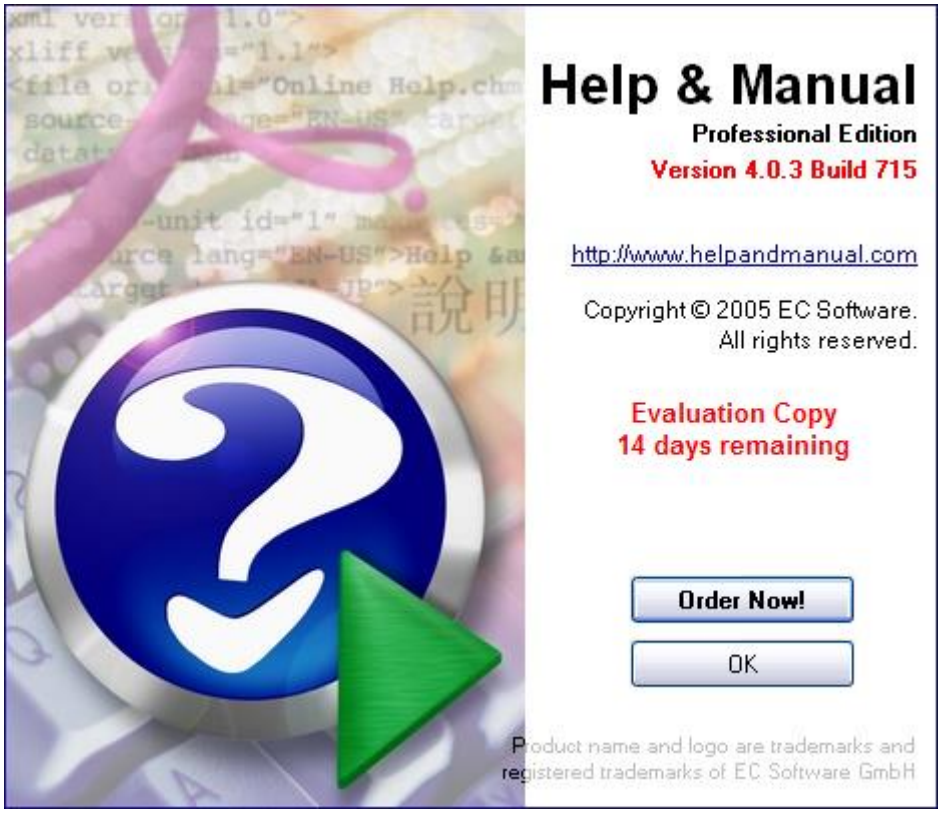
Aquí estoy una vez más con vosotros, con quién mejor? jeje.. en este caso vamos a mirar de superar la protección de un programa que utilizaremos para crear nuestros propios documentos en formato CHM (cortesía de Red@Hawk, muchas gracias amigo) a quien quisiera dedicar especialmente el tute, además como no del resto de la lista por su puesto. Pero no vendamos la piel del oso antes de cazarlo porque todavía no tengo la solución, es más ni lo miré, lo acabo de instalar en la máquina, hoy es día 2 de Enero de 2006 y son exactamente las 7:59h de la mañana ¿prontito eh? pero los hay más madugadores, estaba leyendo los emails recientes de la lista y a estas horas siguen llegando ;-)

Bueno conocido el rival pasaremos a una inspección como las de Hacienda Pública, para estos menesteres tenemos un ejército bien formadito de aplicaciones que nos hacen el cracking más fácil, entremos en batalla con el primero de ellos...

Borland Delphi 4.0 - 5.0

Pues sí acertaron, hablábamos de PEiD 0.94, un escaneador de archivos de lo mejorcito, tenemos también RDG packer 0.5.8 que va muy bien pero no lo necesitamos, es un ejecutable sin empakar compilado en Delphi (de la casa Borland) la versión no la saca bien pero tampoco importa, he de deciros que para este trabajo contaremos con el último Ollydbg que apareció en la lista Ollydbg_full o algo así (sorry, no lo recuerddo...) que va plagado de plugins, muchos ni los necesitaremos pero más vale que sobre que no que falte como se suele decir aquí ;-)

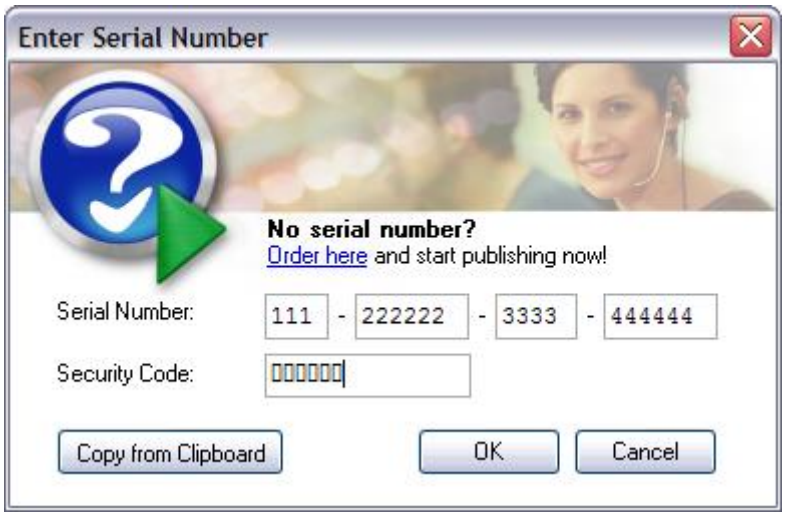
El siguiente punto será pues un primer contacto con la pieza, hay que tener presente que nos ha metido varios ejecutables en la carpeta de instalación, nosotros trabajaremos de momento con "HelpMan.exe" que parece ser el principal de todos, más que nada por su nombre y por esto...



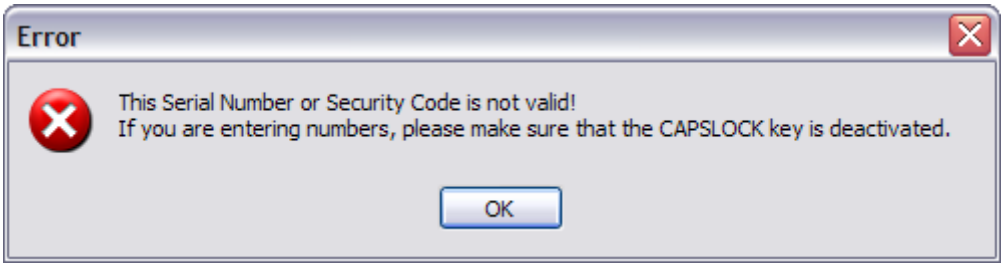
pedazo de nag verdad? el resto de ejecutables no la tienen así que ya es un distintivo que lo diferencia del resto de su família, si pulsamos "Order Now" nos vamos a Internet de cabeza y no nos interesa, de momento aceptamos "OK" y aparece en pantalla la verdadera aplicación acompañada de esta ventanita:

que no sé si es cosa del Trial o es que siemre va a salir ¿? de todos modos como no me hace mucha gracia tener que estar cerrándola cada vez mejor la deshabilitaremos ¿cómo? todavía no lo sé pero lo haremos espero ;-)

Una vez en marcha tenemos una bonita opción que se esconde en el menú "Help" llamada "Enter Serial Number..." jeje.. esto sí nos gusta más, si picamos en ella se abre una ventanita de registro donde introducir nuestros datos:



El Secyrity Code aunque no se vea es "121212" lo que encriptado (lo hace automático) si aceptamos ahora el programa se enfada y ocurre lo que tenía que ocurrir:



Así que invalid serial no? perfecto... cerramos la aplicación y nos vamos al Olly, abrimos el Ollydbg_full, al abrirlo no sé porqué me aparece un cartel cada vez indicando que no encuentra la librería CC3260MT.DLL creo recordar que a alguien también le pasaba, si lo solucionó me ayudaría bastante que me dijera como lo ha hecho, de momento nosotros seguimos.. aceptamos el cartelito y Olly queda en pantalla, arrastramos el ejecutable en cuestión que vamos a estudiar "HelpMan.exe" y queda detenido en el EntryPoint del mismo:

Address	Hex dump	Disassembly	Comment
008B4698	55	PUSH EBP	
008B4699	8BEC	MOV EBP,ESP	
008B469B	83C4 F4	ADD ESP,-0C	
008B469E	53	PUSH EBX	
008B469F	B8 E8388B00	MOV EAX,HelpMan.008B38E8	
008B46A4	E8 F731B5FF	CALL HelpMan.004078A0	
008B46A9	8B1D 8C2B8D00	MOV EBX,DWORD PTR DS:[8D2B8C]	HelpMan.008D3834
008B46AF	8B03	MOV EAX,DWORD PTR DS:[EBX]	
008B46B1	E8 6A06BBFF	CALL HelpMan.00464D20	
008B46B6	8B0B	MOV ECX,DWORD PTR DS:[EBX]	

Ahí estamos en :8B4698 he de decir que el .exe tiene un tamaño considerable, pulsamos F9 y dejamos correr la aplicación, aparece la nag de antes, en este momento pausamos Olly con F12 y miramos la pila a ver qué nos dice:

0012FF00	77D13C6B	RETURN to user32.77D13C6B
0012FF04	0046548C	RETURN to HelpMan.0046548C from <JMP.&user32.WaitMessage>
0012FF08	0012FF5C	Pointer to next SEH record
0012FF0C	004654A7	SE handler
0012FF10	0012FF30	
0012FF14	005A0000	HelpMan.005A0000
0012FF18	77F417E6	RETURN to ntdll.77F417E6 from ntdll.77F68C4E
0012FF1C	016B10FC	
0012FF20	00000000	
0012FF24	00000000	
0012FF28	015A0000	
0012FF2C	016B10FC	
0012FF30	0012FFA8	
0012FF34	00464C40	RETURN to HelpMan.00464C40 from HelpMan.004653B8
0012FF38	00000000	
0012FF3C	0000C0D9	
0012FF40	00000001	
0012FF44	00000000	
0012FF48	00367DBF	ASCII 3F,"oE??pE??qE??rE??sE??tE??uE??vE??wE??xE??yE??zE??{E??|E??}E??
0012FF4C	0000033C	
0012FF50	00000197	
0012FF54	008D3834	HelpMan.008D3834

Esto es lo que vemos al pausar Olly, varios "Return to..." que nos pueden ayudar, aunque sólo uno de ellos volverá al ejecutable, justamente a :464C40 si nos situamos sobre esta línea y pulsamos Enter caemos en la parte del código referenciada:

Address	Hex dump	Disassembly	Comment
00464C2E	E8 41FFFFFF	CALL HelpMan.00464B74	
00464C33	84C0	TEST AL,AL	
00464C35	75 09	JNZ SHORT HelpMan.00464C40	
00464C37	8BD4	MOV EDX,ESP	
00464C39	8BC3	MOV EAX,EBX	
00464C3B	E8 78070000	CALL HelpMan.004653B8	
00464C40	83C4 1C	ADD ESP,1C	;aquÝ vuelve..
00464C43	5B	POP EBX	
00464C44	C3	RETN	

Vemos el call responsable de todo y un JNZ anterior que de producirse eitaría pasar por éste, hmmm.. parece interesante, pongamos un BP sobre el CALL que lo controla:

Address	Hex dump	Disassembly	Comment
00464C2C	8BC3	MOV EAX,EBX	
00464C2E	E8 41FFFFFF	CALL HelpMan.00464B74	
00464C33	84C0	TEST AL,AL	
00464C35	75 09	JNZ SHORT HelpMan.00464C40	
00464C37	8BD4	MOV EDX,ESP	

Y tenemos una trampa preparada ;-) Pero no nos conformemos, tenemos a Olly pausado recodad y en la pila mucha información, volvemos a ella pero ahora hacemos un scroll hacia abajo para que muestre más direcciones, y una que nos interesa es esta:

```
0012FF50  00000197
0012FF54  008D3834  HelpMan.008D3834
0012FF58  00461730  RETURN to HelpMan.00461730 from HelpMan.00464C24
0012FF5C  0012FF68  Pointer to next SEH record
0012FF60  004617B3  SE handler
0012FF64  0012FFA8
```

La que está seleccionada puede interesar, doble clic sobre ella y vemos esto:

```
Address   Hex dump   Disassembly                          Comment
00461730  .  8B03    MOV EAX,DWORD PTR DS:[EBX]           ;aqui salimos..
00461732  .  80B8 8C00000(  CMP BYTE PTR DS:[EAX+8C],0
00461739  .v 74 0F   JE SHORT HelpMan.0046174A
0046173B  .  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
0046173E  .  C780 3402000(  MOV DWORD PTR DS:[EAX+234],2
00461748  .v EB 14   JMP SHORT HelpMan.0046175E
0046174A  >  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
0046174D  .  83B8 3402000(  CMP DWORD PTR DS:[EAX+234],0
00461754  .v 74 08   JE SHORT HelpMan.0046175E
00461756  .  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00461759  .  E8 0EFCFFFF  CALL HelpMan.0046136C
0046175E  >  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00461761  .  8B80 3402000(  MOV EAX,DWORD PTR DS:[EAX+234]
00461767  .  85C0    TEST EAX,EAX
00461769  .^ 74 BE   JE SHORT HelpMan.00461729
0046176B  .  8945 F8 MOV DWORD PTR SS:[EBP-8],EAX
0046176E  .  6A 00   PUSH 0
```

parece el típico buclecillo en el que deja la nag en pantalla a la espera de que hagamos algo con ella (pulsar algún botón..) para cerciorarnosdejamos un BP puesto al final, de esta manera:

```
Address   Hex dump   Disassembly                          Comment
0046174A  >  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
0046174D  .  83B8 3402000(  CMP DWORD PTR DS:[EAX+234],0
00461754  .v 74 08   JE SHORT HelpMan.0046175E
00461756  .  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00461759  .  E8 0EFCFFFF  CALL HelpMan.0046136C
0046175E  >  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00461761  .  8B80 3402000(  MOV EAX,DWORD PTR DS:[EAX+234]
00461767  .  85C0    TEST EAX,EAX
00461769  .^ 74 BE   JE SHORT HelpMan.00461729
0046176B  .  8945 F8 MOV DWORD PTR SS:[EBP-8],EAX
0046176E  .  6A 00   PUSH 0
```

De momento dejamos el otro BP puesto también, ahora dejamos correr nuevamente la aplicación con F9 pero el BP de :464C2E no nos deja en paz ¡! no para de saltar, entonces lo borramos y dejamos únicamente el del final del loop, aceptamos la nag y Olly para justo al final del mismo, si nos finamos vemos que EAX==1, esto de momento no importa, vamos retrocediendo con Ctrl+F2 hasta llegar a esta interesante zona:

```
Address   Hex dump   Disassembly                          Comment
008B46E2  .  A1 242C8D00  MOV EAX,DWORD PTR DS:[8D2C24]
008B46E7  .  8B00    MOV EAX,DWORD PTR DS:[EAX]
008B46E9  .  8078 5D 00  CMP BYTE PTR DS:[EAX+5D],0       ;ojo a esta comparacion..
008B46ED  .v 74 29   JE SHORT HelpMan.008B4718
008B46EF  .  E8 98E2B4FF  CALL HelpMan.0040298C
008B46F4  .  48      DEC EAX
008B46F5  .v 0F8F 8300000(  JG HelpMan.008B477E
008B46FB  .  A1 E0258D00  MOV EAX,DWORD PTR DS:[8D25E0]
008B4700  .  8B00    MOV EAX,DWORD PTR DS:[EAX]
008B4702  .  E8 2DCEBAFF  CALL HelpMan.00461534
008B4707  .  A1 E0258D00  MOV EAX,DWORD PTR DS:[8D25E0]
008B470C  .  8B00    MOV EAX,DWORD PTR DS:[EAX]
008B470E  .  8B10    MOV EDX,DWORD PTR DS:[EAX]
008B4710  .  FF92 8000000(  CALL DWORD PTR DS:[EDX+80]
008B4716  .v EB 66   JMP SHORT HelpMan.008B477E
008B4718  >  A1 E0258D00  MOV EAX,DWORD PTR DS:[8D25E0]
008B471D  .  8B00    MOV EAX,DWORD PTR DS:[EAX]
008B471F  .  8B80 D002000(  MOV EAX,DWORD PTR DS:[EAX+2D0]
008B4725  .  B2 01   MOV DL,1
008B4727  .  E8 D800B9FF  CALL HelpMan.00444804
008B472C  .  A1 E0258D00  MOV EAX,DWORD PTR DS:[8D25E0]
008B4731  .  8B00    MOV EAX,DWORD PTR DS:[EAX]
008B4733  .  8B10    MOV EDX,DWORD PTR DS:[EAX]
008B4735  .  FF92 D800000(  CALL DWORD PTR DS:[EDX+D8]   ;aqui la nag..
008B473B  .  A1 242C8D00  MOV EAX,DWORD PTR DS:[8D2C24]
008B4740  .  8B00    MOV EAX,DWORD PTR DS:[EAX]
```

Para revisar bien este trozo de código pondremos un BP al inicio del mnismo:

```
Address   Hex dump   Disassembly                          Comment
008B46E7  .  8B00    MOV EAX,DWORD PTR DS:[EAX]
008B46E9  .  8078 5D 00  CMP BYTE PTR DS:[EAX+5D],0       ;ojo a esta comparacion..
008B46ED  .v 74 29   JE SHORT HelpMan.008B4718
008B46EF  .  E8 98E2B4FF  CALL HelpMan.0040298C
008B46F4  .  48      DEC EAX
008B46F5  .v 0F8F 8300000(  JG HelpMan.008B477E
008B46FB  .  A1 E0258D00  MOV EAX,DWORD PTR DS:[8D25E0]
```

Y reiniciamos Olly con Ctrl+F2, pulsamos F9 para dejar correr la aplicación, al poco Olly para, estamos en la dirección :8B46E9 donde se está verificando el valor de [EAX+5D] para saber a qué dirección representa y el valor que esconde lo seguiremos con la opción "Follow in dump->Memory Address" y así se verá en la ventana de datos (ventana de dump) ambos valores:

```
Address   Hex dump                                          ASCII
016C4CD1  00 00 00 E0 4D 6C 01 F4 4D 6C 01 28 00 00 00 93  ...óMl0¶Ml0(...ô
016C4CE1  01 00 00 0E 00 00 00 8C 69 6C 01 00 00 00 00 00  0..♫...îil0.....
016C4CF1  00 00 00 FF FF FF FF 00 00 00 00 16 00 00 00 74  ...    .......t
016C4D01  4C 6C 01 00 33 00 00 00 00 00 00 00 00 00 00 52  Ll0.3..........R
016C4D11  00 00 00 01 00 00 00 39 00 00 00 43 52 43 3D 5C  ...0...9...CRC=\
```

Ahí lo tenemos en pantalla, la dirección que representa es [16C4CD1] y su valor es 00 (byte bajo) debemos tener en cuenta que no es una dirección que esté dentro del ejecutable sino una nueva sección creada en tiempo de ejecución al más puro estilo Armadillo, jeje.. para qué tanto rollo? hmmm.. a que nos estamos acercando ;-) de momento probaremos la solución fácil cambiaremos el valor de dicho flag a 01 a ver:

```
Address   Hex dump                                          ASCII
016C4CD1  01 00 00 E0 4D 6C 01 F4 4D 6C 01 28 00 00 00 93  ☺..óMl0¶Ml0(...ô
016C4CE1  01 00 00 0E 00 00 00 8C 69 6C 01 00 00 00 00 00  0..♫...îil0.....
016C4CF1  00 00 00 FF FF FF FF 00 00 00 00 16 00 00 00 74  ...    .......t
016C4D01  4C 6C 01 00 33 00 00 00 00 00 00 00 00 00 00 52  Ll0.3..........R
016C4D11  00 00 00 01 00 00 00 39 00 00 00 43 52 43 3D 5C  ...0...9...CRC=\
```

Lo tenemos ya modificado, ahora si dejamos correr la aplicación con F9 veremos que entramos sin ver la nag, jeje.. aunque seguimos en "Modo Trial" por su puesto.

Bien llegados a este punto tenemos dos caminos: o bien seguir investigando el uso de las variables que utiliza el programa o bien tratar de obtener un serial válido, en prinmer lugar elegiremos la segunda opción: Obtener un serial válido, veremos si es posible, sabemos que al introducir un serial no válido muestra un mensaje de error, pues por aquí entraremos. si ponemos un "bp MessageBoxA" no sacaremos nada, posiblemente utilice alguna como DialogBox o algo así pero bueno cuando esté el mensaje de error en pantalla escribimos en el commandbar:

```
Command  bp DestroyWindow
```

Y aceptamos, si Olly no se queja el BP estará puesto, ahora cerramos el mensaje y para:

```
Address   Hex dump     Disassembly          Comment
77D1A185  B8 63110000  MOV EAX,1163
77D1A18A  BA 0003FE7F  MOV EDX,7FFE0300
77D1A18F  FFD2         CALL EDX
77D1A191  C2 0400      RETN 4
```

Paramos en la API, retrocedemos con Ctrl+F2 hasta el código del .exe pero antes quitamos el BP que ya no nos hace más falta, tras superar varios RET's llegamos a la zona donde está el tema del registro, es esta:

```
Address   Hex dump     Disassembly                  Comment
0087F9C1  .^ EB F0     JMP SHORT HelpMan.0087F9B3
0087F9C3  . 807D FB 00 CMP BYTE PTR SS:[EBP-5],0
0087F9C7  .v 74 33     JE SHORT HelpMan.0087F9FC
0087F9C9  . 6A 00      PUSH 0                       Arg1 = 00000000
0087F9CB  . 66:8B0D B4FB: MOV CX,WORD PTR DS:[87FBB4]
0087F9D2  . B2 03      MOV DL,3
0087F9D4  . B8 C0FB8700 MOV EAX,HelpMan.0087FBC0    ASCII "Thank you for entering the seri
0087F9D9  . E8 C2D4BBFF CALL HelpMan.0043CEA0       HelpMan.0043CEA0
0087F9DE  . 83F8 06    CMP EAX,6
0087F9E1  .v 75 0A     JNZ SHORT HelpMan.0087F9ED
0087F9E3  . 8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
0087F9E6  . C740 0C 0500 MOV DWORD PTR DS:[EAX+C],5
0087F9ED  > 8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
0087F9F0  . C780 3402000 MOV DWORD PTR DS:[EAX+234],1
0087F9FA  .v EB 42     JMP SHORT HelpMan.0087FA3E
0087F9FC  > 6A 00      PUSH 0                       Arg1 = 00000000
0087F9FE  . 66:8B0D 8CFA: MOV CX,WORD PTR DS:[87FA8C]
0087FA05  . B2 01      MOV DL,1
0087FA07  . B8 64FC8700 MOV EAX,HelpMan.0087FC64    ASCII "Could not save serial number in
0087FA0C  . E8 8FD4BBFF CALL HelpMan.0043CEA0       HelpMan.0043CEA0
0087FA11  .v EB 2B     JMP SHORT HelpMan.0087FA3E
0087FA13  > 6A 00      PUSH 0                       Arg1 = 00000000
0087FA15  . 66:8B0D 8CFA: MOV CX,WORD PTR DS:[87FA8C]
0087FA1C  . B2 01      MOV DL,1
0087FA1E  . B8 1CFD8700 MOV EAX,HelpMan.0087FD1C    ASCII "This Serial Number or Security
0087FA23  . E8 78D4BBFF CALL HelpMan.0043CEA0       HelpMan.0043CEA0
0087FA28  . 8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
0087FA2B  . 8378 0C 05 CMP DWORD PTR DS:[EAX+C],5
0087FA2F  .v 7C 0D     JL SHORT HelpMan.0087FA3E
0087FA31  . 8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
```

Vemos todos los mensajes que tiene preparados, inclusive el de felicitación, pero no acaba ahí pues el call es mucho más largo de lo que en realidad vemos en la imagen, así que subimos hasta encontrar su inicio y le ponemos un BP como en la imagen:

```
Address   Hex dump        Disassembly                          Comment
0087F5A4  .  55           PUSH EBP                             ;inicio_rutina
0087F5A5  .  8BEC         MOV EBP,ESP
0087F5A7  .  B9 08000000  MOV ECX,8
0087F5AC  >  6A 00        PUSH 0
0087F5AE  .  6A 00        PUSH 0
0087F5B0  .  49           DEC ECX
0087F5B1  .^ 75 F9        JNZ SHORT HelpMan.0087F5AC
0087F5B3  .  8945 FC      MOV DWORD PTR SS:[EBP-4],EAX
0087F5B6  .  33C0         XOR EAX,EAX
0087F5B8  .  55           PUSH EBP
0087F5B9  .  68 80FA8700  PUSH HelpMan.0087FA80
0087F5BE  .  64:FF30      PUSH DWORD PTR FS:[EAX]
0087F5C1  .  64:8920      MOV DWORD PTR FS:[EAX],ESP
0087F5C4  .  8D55 F0      LEA EDX,DWORD PTR SS:[EBP-10]
0087F5C7  .  8B45 FC      MOV EAX,DWORD PTR SS:[EBP-4]
0087F5CA  .  8B80 D802000( MOV EAX,DWORD PTR DS:[EAX+2D8]
0087F5D0  .  E8 1753BCFF  CALL HelpMan.004448EC
0087F5D5  .  8B45 F0      MOV EAX,DWORD PTR SS:[EBP-10]
0087F5D8  .  E8 7B4BB8FF  CALL HelpMan.00404158
0087F5DD  .  8B55 FC      MOV EDX,DWORD PTR SS:[EBP-4]
```

Allí lo tenemos bien puesto, ahora volvemos a aceptar y el Olly para en el BP, vamos traceando a ver qué vemos, pues no hay que tracear mucho para comenzar a visualizar cosas, en primer lugar en este CALL:

```
Address   Hex dump        Disassembly                          Comment
0087F5D5  .  8B45 F0      MOV EAX,DWORD PTR SS:[EBP-10]        ;parte1 del serial..
0087F5D8  .  E8 7B4BB8FF  CALL HelpMan.00404158
0087F5DD  .  8B55 FC      MOV EDX,DWORD PTR SS:[EBP-4]
0087F5E0  .  8B92 D802000( MOV EDX,DWORD PTR DS:[EDX+2D8]
```

```
Registers (FPU)
EAX 01715EAC ASCII "111"
ECX 0012F5FC
EDX 00402370 HelpMan.00402370
EBX 01765B50
ESP 0012F5FC
```

si miramos en los registros vemos la primera parte del serial, recordad que pusimos:

111-222222-3333-444444

Poco después del CALL una comprobación, mira que el número de dígitos del primer bloque del serial sean justamente 3 como es el caso, esto lo mira aquí:

```
Address   Hex dump        Disassembly                          Comment
0087F5E0  .  8B92 D802000( MOV EDX,DWORD PTR DS:[EDX+2D8]
0087F5E6  .  3B82 F001000( CMP EAX,DWORD PTR DS:[EDX+1F0]      ;mira num digitos..
0087F5EC  .v 0F85 9C00000( JNZ HelpMan.0087F68E
0087F5F2  .  8D55 EC      LEA EDX,DWORD PTR SS:[EBP-14]
```

como sabemos que quiere tres? fácil si miramos la ventanita auxiliar que para algo está no? echamos un vistazo y vemos qué es lo que está comparando con EAX:

```
DS:[017641F4]=00000003
EAX=00000003
```

Ahí lo vemos compara un 3 (número de dígitos puestos) con un 3 (número de dígitos que debe contener el bloque), bien pues este mismo esquema se repite para los bloques 2ª, 3ª y 4ª no lo vamos a repetir todo claro está sólo daremos la dirección de la CMP:

<u>2° bloque</u>:

```
Address   Hex dump        Disassembly                          Comment
0087F60E  .  8B92 DC02000( MOV EDX,DWORD PTR DS:[EDX+2DC]
0087F614  .  3B82 F001000( CMP EAX,DWORD PTR DS:[EDX+1F0]      ;CMP parte2..
0087F61A  .v 75 72        JNZ SHORT HelpMan.0087F68E
0087F61C  .  8D55 E8      LEA EDX,DWORD PTR SS:[EBP-18]
```

<u>3° bloque</u>:

```
Address   Hex dump        Disassembly                          Comment
0087F638  .  8B92 E002000( MOV EDX,DWORD PTR DS:[EDX+2E0]
0087F63E  .  3B82 F001000( CMP EAX,DWORD PTR DS:[EDX+1F0]      ;CMP parte3..
0087F644  .v 75 48        JNZ SHORT HelpMan.0087F68E
0087F646  .  8D55 E4      LEA EDX,DWORD PTR SS:[EBP-1C]
```

<u>4° bloque</u>:

```
Address   Hex dump        Disassembly                          Comment
0087F662  .  8B92 E402000( MOV EDX,DWORD PTR DS:[EDX+2E4]
0087F668  .  3B82 F001000( CMP EAX,DWORD PTR DS:[EDX+1F0]      ;CMP parte4..
0087F66E  .v 75 1E        JNZ SHORT HelpMan.0087F68E
0087F670  .  8D55 E0      LEA EDX,DWORD PTR SS:[EBP-20]
```

Y ya los tenemos todos, en caso de no coincidir el número de dígitos esperado en cada caso con el introducido iremos directos a :87F68E donde nos pelarán claro, si superamos todas estas CPM's con éxito seguiremos traceando pero aún hay más comprobaciones, esta que viene es otra muy similar pero para el "Security Code" aquí está:

```
Address   Hex dump     Disassembly                      Comment
0087F67C  .  E8 6B52BCFF  CALL HelpMan.004448EC
0087F681  .  8B45 E0      MOV EAX,DWORD PTR SS:[EBP-20]
0087F684  .  E8 CF4AB8FF  CALL HelpMan.00404158
0087F689  .  83F8 04      CMP EAX,4                      ;de Security Code..
0087F68C  .v 7D 1A        JGE SHORT HelpMan.0087F6A8
0087F68E  >  6A 00        PUSH 0                         Arg1 = 00000000
0087F690  .  66:8B0D 8CFA MOV CX,WORD PTR DS:[87FA8C]
0087F697  .  B2 02        MOV DL,2
0087F699  .  B8 98FA8700  MOV EAX,HelpMan.0087FA98       ASCII "Serial Number and/o
0087F69E  .  E8 FDD7BBFF  CALL HelpMan.0043CEA0          HelpMan.0043CEA0
0087F6A3  .v E9 96030000  JMP HelpMan.0087FA3E
0087F6A8  >  33D2         XOR EDX,EDX
```

Vemos que si el número de dígitos no es igual o mayor que 4 nos saca el mensaje de error directamente, jeje.. pusimos 6 así que pasamos bien esta también ;-) hace lo pripio con el "Serial Number" en este otro CALL:

```
Address   Hex dump     Disassembly                      Comment
0087F700  .  8B80 1003000 MOV EAX,DWORD PTR DS:[EAX+310]
0087F706  .  E8 4D4AB8FF  CALL HelpMan.00404158
0087F70B  .  83F8 16      CMP EAX,16                     ;de Serial Number..
0087F70E  .v 75 2A        JNZ SHORT HelpMan.0087F73A
0087F710  .  8D4D DC      LEA ECX,DWORD PTR SS:[EBP-24]
0087F713  .  A1 242C8D00  MOV EAX,DWORD PTR DS:[8D2C24]
```

Mira el serial por completo y mira que tenga 16h dígitos (o sea 23 decimal) si no los tiene ni que decir que nos pega un estacazo, pero como los tiene seguimos, jeje.. poco después llega a una zona más que interesante diría yo, veámosla primero:

```
Address   Hex dump     Disassembly                      Comment
0087F722  .  E8 F95CB9FF  CALL HelpMan.00415420
0087F727  .  8B55 DC      MOV EDX,DWORD PTR SS:[EBP-24]  ;serial_valido..
0087F72A  .  8B45 FC      MOV EAX,DWORD PTR SS:[EBP-4]
0087F72D  .  8B80 1003000 MOV EAX,DWORD PTR DS:[EAX+310] ;serial_falso..
0087F733  .  E8 304BB8FF  CALL HelpMan.00404268          ;los compara!!
0087F738  .v 74 04        JE SHORT HelpMan.0087F73E
0087F73A  >  33C0         XOR EAX,EAX
0087F73C  .v EB 02        JMP SHORT HelpMan.0087F740
0087F73E  >  B0 01        MOV AL,1
0087F740  >  8845 FA      MOV BYTE PTR SS:[EBP-6],AL
0087F743  .  807D FA 00   CMP BYTE PTR SS:[EBP-6],0
0087F747  .v 75 52        JNZ SHORT HelpMan.0087F79B
```

Pone un puntero en EAX a nuestro serial en :87F72D pero antes pone un puntero en EDX a un serial ¿el válido? en :87F727 luego se mete en el CALL y salta o no en función de si son iguales, como resultado activa el flag de éxito (EAX==1) o el de error (EAX==0) pasando dicho valor a la variable local [EBP-6] que después verifica en :87F743 y en caso de no ser 0 salta evitando ejecutar gran parte de la rutina, la pregunta es ¿será este el serial válido? veamos los registros primero:

```
Registers (FPU)
EAX 017664EC ASCII "111-222222-3333-444444"
ECX 00000001
EDX 01766510 ASCII "HM4-DT4391-SL5T-K6RMVX"
EBX 01765B50
ESP 0012F5F0
```

Hombre MALO no parece ser pero quizá esté Blackisteado ¿qué es esto? rápidamente diríamos que algunos programadores están hartos de ver cómo sus copias son pirateadas a través de seriales válidos que circulan libremente por Internet, su reacción es poner estos códigos en una lista negra para "castigar" de alguna manera a aquellos que los utilicen, sus resultados pueden ser imprevisibles...

En fin que de momento tracearemos la rutina un poquito más, jeje.. no tejgo ganas de juerga, si seguimos traceando vemos como code el Security Code y le aplica ciertas operaciones (al menos eso parece no lo miré mucho de momento) y al final compara el resultado con parte del propio SecCode, es aquí:

```
Address   Hex dump     Disassembly                      Comment
0087F78C  .  8B55 CC      MOV EDX,DWORD PTR SS:[EBP-34]
0087F78F  .  58           POP EAX
0087F790  .  E8 D34AB8FF  CALL HelpMan.00404268          ;vuelve a comparar..
0087F795  .v 74 04        JE SHORT HelpMan.0087F79B
0087F797  .  33C0         XOR EAX,EAX
0087F799  .v EB 02        JMP SHORT HelpMan.0087F79D
0087F79B  >  B0 01        MOV AL,1
```

En este punto si miramos los registros como hicimos en el anterior punto donde verificaba ambos seriales veremos lo que está comparando:

```
Registers (FPU)
EAX 0176655C ASCII "1212"
ECX 00000203
EDX 017181D0 ASCII "515"
EBX 01765B50
```

En este caso nuestro SecCode era recuerden "121212" del que coge una parte (no sabemos cual de momento) y la compara con "515" si son iguales realiza la misma jugada de antes activa AL con un 1 sinó deja EAX==0, como no es nuestro caso queda lógicamente EAX==0, seguimos y vemos que compara el contenido de [EBP-6] con 0 ya veremos a qué posición de memoria se refiera luego, por ahora decir que el salto no parece muy bueno, veamos que mensaje evitamos al saltar ;-(

```
Address   Hex dump    Disassembly                          Comment
0087F7D9  .^ EB E0    JMP SHORT HelpMan.0087F7BB
0087F7DB  .  807D FA 00  CMP BYTE PTR SS:[EBP-6],0         ;verifica el flag..
0087F7DF  .v 0F84 A700000  JE HelpMan.0087F88C             ;y salta..
0087F7E5  .  A1 242C8D00  MOV EAX,DWORD PTR DS:[8D2C24]
0087F7EA  .  8B00     MOV EAX,DWORD PTR DS:[EAX]
```

Este era el salto y ahora veamos el código que no se ejcuta a causa suya:

```
Address   Hex dump    Disassembly                          Comment
0087F85B  >  6A 00    PUSH 0                               Arg1 = 00000000
0087F85D  .  66:8B0D 8CFA  MOV CX,WORD PTR DS:[87FA8C]
0087F864  .  B2 03    MOV DL,3
0087F866  .  B8 38FB8700  MOV EAX,HelpMan.0087FB38         ASCII "Thank you for evalu
0087F86B  .  E8 30D6BBFF  CALL HelpMan.0043CEA0            HelpMan.0043CEA0
0087F870  .  8B45 FC  MOV EAX,DWORD PTR SS:[EBP-4]
0087F873  .  C740 0C 0500  MOV DWORD PTR DS:[EAX+C],5
0087F87A  .  8B45 FC  MOV EAX,DWORD PTR SS:[EBP-4]
0087F87D  .  C780 3402000  MOV DWORD PTR DS:[EAX+234],1
0087F887  .v E9 B2010000  JMP HelpMan.0087FA3E
0087F88C  >  807D FB 00  CMP BYTE PTR SS:[EBP-5],0
0087F890  .v 0F84 7D01000  JE HelpMan.0087FA13
0087F896  .  C645 FB 00  MOV BYTE PTR SS:[EBP-5],0
0087F89A  .  B2 01    MOV DL,1
```

Ahí lo vemos el "Thanks, bla, bla.." que parece el mensaje de felicitación, si nos ponemos sobre la variable [EBP-6] y hacemos un "Follow in dump->Memory Address" veremos

```
Address   Hex dump                                         ASCII
0012F642  00 00 04 C0 74 01 AC F7 12 00 54 5C 44 00 50 5B  ..♦└t0¼-♣.T\D.P[
0012F652  76 01 99 11 43 00 78 11 43 00 E3 12 43 00 C8 F9  v00◄C.x◄C.∂♣C.╚·
0012F662  12 00 C8 F9 12 00 50 5B 76 01 C0 5A 44 00 C8 F9  ♣.╚·♣.P[v0└ZD.╚·
0012F672  12 00 C8 F9 12 00 50 5B 76 01 17 86 44 00 C8 F9  ♣.╚·♣.P[v0♣▬D.╚·
```

representa a [12F642] y su contenido es un 0 por esto salta, si seguimos traceando la rutina llegamos a otra comparación y posterior salto que evita otro posible mensaje de felicitación, esta es la CMP:

```
Address   Hex dump    Disassembly                          Comment
0087F87D  .  C780 3402000  MOV DWORD PTR DS:[EAX+234],1
0087F887  .v E9 B2010000  JMP HelpMan.0087FA3E
0087F88C  >  807D FB 00  CMP BYTE PTR SS:[EBP-5],0         ;otra comparacion..
0087F890  .v 0F84 7D01000  JE HelpMan.0087FA13             ;y mas salto..
0087F896  .  C645 FB 00  MOV BYTE PTR SS:[EBP-5],0
0087F89A  .  B2 01    MOV DL,1
```

Y este el posible mensaje que evita el salto:

```
Address   Hex dump    Disassembly                          Comment
0087F9C3  .  807D FB 00  CMP BYTE PTR SS:[EBP-5],0
0087F9C7  .v 74 33    JE SHORT HelpMan.0087F9FC
0087F9C9  .  6A 00    PUSH 0                               Arg1 = 00000000
0087F9CB  .  66:8B0D B4FB  MOV CX,WORD PTR DS:[87FBB4]
0087F9D2  .  B2 03    MOV DL,3
0087F9D4  .  B8 C0FB8700  MOV EAX,HelpMan.0087FBC0         ASCII "Thank you for entering the
0087F9D9  .  E8 C2D4BBFF  CALL HelpMan.0043CEA0            HelpMan.0043CEA0
```

seguimos pues sin ver nada claro, parece una rutina un tanto atípica y liosa pero con tranquilidad se hacen las cosas, además ese JE anterior nos pone de cabeza en el mensaje de error, está claro que no es bueno, lo siguiente que haremos antes de seguir estrujándonos el tarro será lo más sencillo ¡claro! probar el serial que hemos pillado antes en el CALL que había en :87F733 y cruzar los dedos ;-)

Aceptamos y nos fijamos que ahora en el CALL que compara salta...

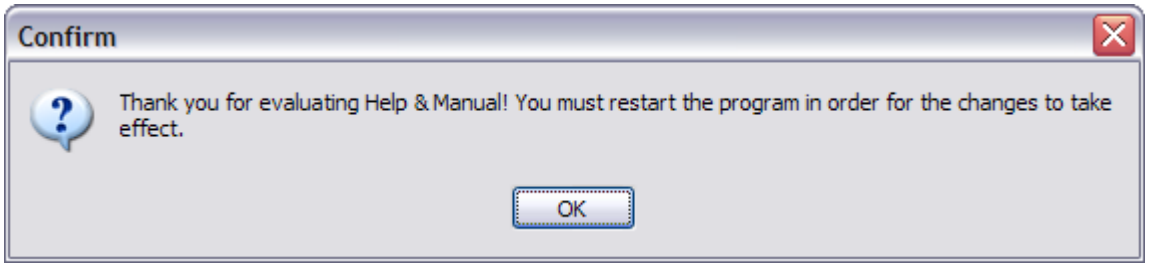
Address	Hex dump	Disassembly	Comment
0087F72D	. 8B80 1003000(	MOV EAX,DWORD PTR DS:[EAX+310]	;serial_falso..
0087F733	. E8 304BB8FF	CALL HelpMan.00404268	;los compara!!
0087F738	.v 74 04	JE SHORT HelpMan.0087F73E	
0087F73A	> 33C0	XOR EAX,EAX	
0087F73C	.v EB 02	JMP SHORT HelpMan.0087F740	
0087F73E	> B0 01	MOV AL,1	
0087F740	> 8845 FA	MOV BYTE PTR SS:[EBP-6],AL	

El JE se produce cosa que antes no ocurría, se pone AL==1 y se guarda AL en [EBP-6] claro la variable que testeaba después y nos enviaba al error, jeje.. está claro que vamos bien, si seguimos hasta la citada comprobación tendremos nuestra recompensa ;-)

Address	Hex dump	Disassembly	Comment
0087F7D4	.^ E9 AF40B8FF	JMP HelpMan.00403888	
0087F7D9	.^ EB E0	JMP SHORT HelpMan.0087F7BB	
0087F7DB	. 807D FA 00	CMP BYTE PTR SS:[EBP-6],0	;verifica el flag..
0087F7DF	.v 0F84 A700000(	JE HelpMan.0087F88C	;y salta..
0087F7E5	. A1 242C8D00	MOV EAX,DWORD PTR DS:[8D2C24]	
0087F7EA	. 8B00	MOV EAX,DWORD PTR DS:[EAX]	

Si miramos en la ventanita auxiliar veremos que efectivamente el flag está a 1 ;-)

pues nada, dejamos correr la aplicación con F9 y nos sale este mensaje en pantalla:



Nos da las gracias y nos pide que reiniciemos la app, hmmm.. para que querrá reiniciar? no estaba claro que ese era el serial válido ¿? bueno reiniciemos a ver... ejecutamos de nuevo el programa y... de nuevo la nagscreen..!!

Bueno después de todo está claro que algo se nos está escapando, ya sabemos que es una rutina un tanto liosa como hemos dicho antes pero tiene solución eso fijo! reiniciamos Olly y dejamos el BP puesto en el CALL donde compara los seriales, recordemos que después realizaba otra comparación con parte del Security Code (SecCode) veamos si podemos hacer algo con esto ;-)

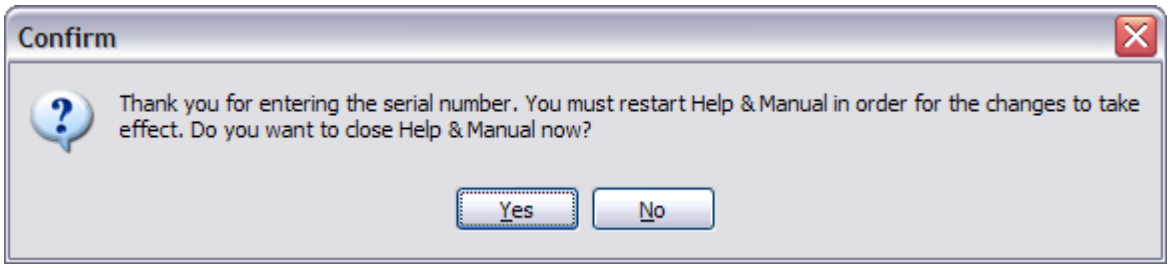
Antes de nada dejaremos un SecCode totalmente reconocible, es decir de nada nos vale el 121212 anterior pues si toma 1212 ¿qué dígitos está cogiendo? para evitar errores pondremos este otro "123456" y ahora sí sabremos con franqueza lo que está mirando, lo ponemos junto a cualquier SecCode y vamos a ver qué dígitos espera encontrar y en qué posiciones, esto lo sabemos poniendo uyn BP en :87F790 y mirando los registros cuando Olly pare allí, esto es lo que vemos:

```
Registers (FPU)
EAX 016CC454 ASCII "3456"
ECX 00000203
EDX 01732268 ASCII "515"
EBX 01764DF0
ESP 0012F5F0
```

Vaya! vuelve a buscar el 515 de antes, pues nada nuestro SecCode será "12515" y veamos que pasa ahora..

```
Registers (FPU)
EAX 0173CBEC ASCII "515"
ECX 00000203
EDX 0170A338 ASCII "515"
EBX 01764DF0
ESP 0012F2F4
```

de momento llegamos con los mismos dígitos a la comparación, pulsamos F9 y...



hombe otro mensaje difrente al anterior aunque nos sigue pidiendo un reinicio ¿? aceptamos el mensaje y él solito se reinicia, lo volvemos a ejecutar y... nos sigue apareciendo la nagscreen!!!!!!!!!

Esto es un horroooooooooooor por más que hacemos siempre nos pilla ;-(

A estas alturas se me ocurrió otra cosa a ver que tal va, sabemos que el serial lo almacena en el registro de Window$ y lo recupera al inicio del programa, de no ser así no tiene lógica que debamos reiniciar para validar el serial, casi he llegado a la conclusión de que la rutina con la que nos hemos peleado está ahí de farol, es una suposición pero no comprendo com hallando un serial que pase los filtros de la misma lego no sea realmente válido, a no ser que la rutina no haga ningún trabajo y esté para despistar, hace como si verifica el serial pero en realidad no vale para nada, esto como digo es pura suposición sino no lo comprendo ¡!

En fin a lo que vamos, sabemos que el programa guarda el serial en algún lugar del HD (digamos que en el registro como el resto de programas) antiguamente lo hacían en los archivos .ini que dejaban en el directorio "x:\Window$ " y "x:\Window$\system32" pero hoy día lo suelen hacer en el registro como digo, accesible desde regedit.exe, bien tenemos regmonitor para verificar esto pero no hará falta de momento, lo que haremos será poner un par de BP's en apis que se utilizan para obtener información del mismo, estas apis son RegQueryValueA y RegQueryValueExA, entonces pondremos un BP en ambas:

El primero de ellos:

Y el segundo:

Ya los tenemos puestos verdad? pues dejamos correr la aplicación con F9 otra vez y a esperar que Olly vaya parando...

```
0012EF1C  746B2215  CALL to RegQueryValueExA from 746B220F
0012EF20  00000088  hKey = 88
0012EF24  746B2E9C  ValueName = "CUAS"
0012EF28  00000000  Reserved = NULL
0012EF2C  0012EF38  pValueType = 0012EF38
0012EF30  0012EF7C  Buffer = 0012EF7C
0012EF34  0012EF3C  pBufSize = 0012EF3C
0012EF38  00000000
```

nada.. F9..

```
0012EF28  746B9D6C  CALL to RegQueryValueExA from 746B9D66
0012EF2C  0000009C  hKey = 9C
0012EF30  746B3BE8  ValueName = "Language Hotkey"
0012EF34  00000000  Reserved = NULL
0012EF38  0012EF4C  pValueType = 0012EF4C
0012EF3C  0012EF78  Buffer = 0012EF78
0012EF40  0012EF60  pBufSize = 0012EF60
```

nada.. F9.. nada.. F9.. nada.. F9... ... eeeeeeeeeeei!!!! mirad:

```
0012FD50  0049F657  CALL to RegQueryValueExA from HelpMan.0049F652
0012FD54  000000D4  hKey = D4
0012FD58  0075815C  ValueName = "SN"
0012FD5C  00000000  Reserved = NULL
0012FD60  0012FD6C  pValueType = 0012FD6C
0012FD64  00000000  Buffer = NULL
0012FD68  0012FD88  pBufSize = 0012FD88
```

pero qué vemos ahí ;-) SN de Serial Number quizá? le seguiremos la pista, en este momento estamos en el código de la API (dentro del kernel en Advapi32) que es una de sus funciones exportadas, lo que necesitamos nosotros ahora es retornar al código del ejecutable desde donde se hizo la llamada, esto lo conseguimos bien a base de traceo con F7 (interminable) ó F8 (poco recomensable) o bien con Ctrl+F9 que va ejecutando instrucciones que hasta encuentra un RET, así lo haremos y caemos aquí:

```
Address   Hex dump    Disassembly                          Comment
0049F64D  .  50       PUSH EAX                             ValueName
0049F64E  .  8B46 04  MOV EAX,DWORD PTR DS:[ESI+4]
0049F651  .  50       PUSH EAX                             hKey
0049F652  .  E8 3583F6FF  CALL <JMP.&advapi32.RegQueryValueExA>  RegQueryValueExA
0049F657  .  85C0     TEST EAX,EAX                         ;aqui salimos..
0049F659  .  0F94C3   SETE BL
0049F65C  .  8B0424   MOV EAX,DWORD PTR SS:[ESP]
0049F65F  .  E8 50F9FFFF  CALL HelpMan.0049EFB4
0049F664  .  8845 00  MOV BYTE PTR SS:[EBP],AL
0049F667  .  8BC3     MOV EAX,EBX
0049F669  .  5A       POP EDX
```

pero esto no nos dice nada, seguimos con el método y caemos en otra llamada similar:

```
0012FD54  0049F81C  CALL to RegQueryValueExA from HelpMan.0049F817
0012FD58  000000D4  hKey = D4
0012FD5C  0075815C  ValueName = "SN"
0012FD60  00000000  Reserved = NULL
0012FD64  0012FD84  pValueType = 0012FD84
0012FD68  016C37C4  Buffer = 016C37C4
0012FD6C  0012FD94  pBufSize = 0012FD94
```

esta vez la llamada se realizó desde :49F817 dentro de nuestro ejecutable, vamos con Ctrl+F9 hasta salir al lugar de la llamada:

```
Address   Hex dump    Disassembly                          Comment
0049F812  .  50       PUSH EAX                             ValueName
0049F813  .  8B43 04  MOV EAX,DWORD PTR DS:[EBX+4]
0049F816  .  50       PUSH EAX                             hKey
0049F817  .  E8 7081F6FF  CALL <JMP.&advapi32.RegQueryValueExA>  RegQueryValueExA
0049F81C  .  85C0     TEST EAX,EAX
0049F81E  .v 74 24    JE SHORT HelpMan.0049F844
0049F820  .  8975 F4  MOV DWORD PTR SS:[EBP-C],ESI
```

estamos a la salida de la API, si miramos los registros comenzamos a ver algo de luz xD

```
Registers (3DNow!)
EAX 00000000
ECX 00000001
EDX 0012FD94
EBX 016B3E5C
ESP 0012FD70
EBP 0012FD88
ESI 0075815C ASCII "SN"
EDI 016C37C4 ASCII "111-111111-1111-111111"
```

jeje.. ya aparece nuestro serial (que no vale para nada pero bueno) en uno de los registros, en concreto EDI tiene un puntero al mismo, parece ser que la primera llamada a RegQueryValueExA era para verificar si la clave existía realmente en el registro, al ver que así es lanza una segunda para recuperar su contenido y este no es más que nuestro serial trampa, jeje.. ahora sólo debemos ser pacientes hasta localizar la zona donde comprueba si es o no válido, si lo logramos podremos forzar a la aplicación a inicializar el flag de registro de modo que correrá en "Modo Completo" pero esto aún no lo hemos conseguido, vamos poco a poco, si vamos superando los RET's que encontremos al paso llegaremos justo al inicio de todo:

```
Address   Hex dump     Disassembly                          Comment
00757EB7  .  84C0       TEST AL,AL
00757EB9  .v 74 31      JE SHORT HelpMan.00757EEC
00757EBB  .  BA 2C817500 MOV EDX,HelpMan.0075812C            ASCII "\SOFTWARE\ECSOFTWARE\Help & Manual 4"
00757EC0  .  33C9       XOR ECX,ECX
00757EC2  .  8B45 EC    MOV EAX,DWORD PTR SS:[EBP-14]
00757EC5  .  E8 3273D4FF CALL HelpMan.0049F1FC
00757ECA  .  84C0       TEST AL,AL
00757ECC  .v 74 1E      JE SHORT HelpMan.00757EEC
00757ECE  .  8D4D B0    LEA ECX,DWORD PTR SS:[EBP-50]
00757ED1  .  BA 5C817500 MOV EDX,HelpMan.0075815C            ASCII "SN"
00757ED6  .  8B45 EC    MOV EAX,DWORD PTR SS:[EBP-14]
00757ED9  .  E8 E677D4FF CALL HelpMan.0049F6C4
00757EDE  .  8B55 B0    MOV EDX,DWORD PTR SS:[EBP-50]
00757EE1  .  8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
```

Ahí nace todo lo visto en esa llamada que dejamos atrás, nosotros eguiremos traceando sin perder detalle, si somos observadores veremos nuestro serial en el registro EAX en este momento, así que atentos que está a punto de ocurrir lo que buscamos, al final llegamos a la zona clave de todo esto:

```
Address   Hex dump     Disassembly                          Comment
00757F81  .  BA 01000000 MOV EDX,1
00757F86  .  E8 D5C3CAFF CALL HelpMan.00404360
00757F8B  .  8D55 F0    LEA EDX,DWORD PTR SS:[EBP-10]
00757F8E  .  8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
00757F91  .  8B40 78    MOV EAX,DWORD PTR DS:[EAX+78]
00757F94  .  E8 AFF9FFFF CALL HelpMan.00757948
00757F99  .  8B55 FC    MOV EDX,DWORD PTR SS:[EBP-4]
00757F9C  .  8B45 F0    MOV EAX,DWORD PTR SS:[EBP-10]
00757F9F  .  8942 7C    MOV DWORD PTR DS:[EDX+7C],EAX        ;modifica [16C4CF0]..
00757FA2  .  85C0       TEST EAX,EAX
00757FA4  .  0F95C0     SETNE AL
00757FA7  .  8B55 FC    MOV EDX,DWORD PTR SS:[EBP-4]
00757FAA  .  8842 5D    MOV BYTE PTR DS:[EDX+5D],AL          ;modifica [16C4CD1]..
00757FAD  .  8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
00757FB0  .  8378 7C 00 CMP DWORD PTR DS:[EAX+7C],0          ;verifica [16C4CF0]..
00757FB4  .v 74 0C      JE SHORT HelpMan.00757FC2
00757FB6  .  8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
00757FB9  .  8B55 F4    MOV EDX,DWORD PTR SS:[EBP-C]
00757FBC  .  8990 8000000( MOV DWORD PTR DS:[EAX+80],EDX
00757FC2  >  8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
00757FC5  .  8078 5D 00 CMP BYTE PTR DS:[EAX+5D],0
00757FC9  .v 74 08      JE SHORT HelpMan.00757FD3
00757FCB  .  8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
00757FCE  .  33D2       XOR EDX,EDX
```

Aquí se decide finalmente si el serial es o no válido, se mete en el CALL de :757F94 donde lo comprueba, como no queremos hacer el KeyGen (de momento ;-d) nos saltamos esta parte sinó el tute sería una biblia, podemos entrar si queremos a echar un vistazo y veremos una larga cadena que parece una llave de encriptación y un loop, después se mete en un CALL, mira si EAX==10, etc, etc., y al final retorna que es a lo que a nosotros nos interesa ahora, cuando sale se dispone a modificar las direcciones de memoria [16C4CF0] y [16C4CD1], este último era del que nos ayudábamos antes para evitar la nag pero ¿el otro? jeje.. parece una réplica, quizá lo tenga por si modificamos malintencionadamente alguno de ellos tener una coipia del valor original y así poder pillarnos, recordad que aún modificando uno de ellos evitábamos la nag pero no nos registrábamos y quizá es por esto, en cualquier caso vemos que va a recuperar un 0 que dejará en EAX en :757F9C así que no nos interesa, necesitamos que deje EAX==1, el caso más sencillo para esto sería modificar el código de esta manera:

```
Address   Hex dump     Disassembly                          Comment
00757F91  .  8B40 78    MOV EAX,DWORD PTR DS:[EAX+78]
00757F94  .  E8 AFF9FFFF CALL HelpMan.00757948
00757F99  .  8B55 FC    MOV EDX,DWORD PTR SS:[EBP-4]
00757F9C     33C0       XOR EAX,EAX
00757F9E     40         INC EAX
00757F9F  .  8942 7C    MOV DWORD PTR DS:[EDX+7C],EAX        ;modifica [16C4CF0]..
00757FA2  .  85C0       TEST EAX,EAX
00757FA4  .  0F95C0     SETNE AL
00757FA7  .  8B55 FC    MOV EDX,DWORD PTR SS:[EBP-4]
00757FAA  .  8842 5D    MOV BYTE PTR DS:[EDX+5D],AL          ;modifica [16C4CD1]..
00757FAD  .  8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
00757FB0  .  8378 7C 00 CMP DWORD PTR DS:[EAX+7C],0          ;verifica [16C4CF0]..
00757FB4  .v 74 0C      JE SHORT HelpMan.00757FC2
00757FB6  .  8B45 FC    MOV EAX,DWORD PTR SS:[EBP-4]
```

Así dejamos un 1 en EAX cuando pasemos por aquí y pensará que el serial es válido, sólo nos queda salvar los cambios, picamos con el botón derecho con las instrucciones modificadas seleccionadas como en la imagen y elegir "Copy to executable->Selection" se abrirá otra pantallita donde pulsamos con el botón derecho y elegimos "Save file" lo llamamos tute.exe de momento, ahora lo ejecutamos y...

SIIIIIIIIIi.....!!!

Entramos sin la nagscreen, en su lugar muestra la misma pantallita pero tipo banner que se va sóla tras un pequeño retardo en la que muestra nuestro serial como válido, jeje.. prueba superada, bueno veremos no me fio ya..

Si vamos al menú "Help" vemos que la opción de introdcir un serial ha desaparecido por completo, ahora sí parece que hemos vencido, bonita acción el jugar con 2 banderas:

[16C4CF0]
[16C4CD1]

si alguna de ellas es modificada la otra almacenará una copia del valor original y nos pillará, menos mal que el programador no es mal tío.. bouuuuuuuff!

Si miramos en el About allí veremos nuestro serial falso que al final aceptó ;-)

Y aquí acaba el tute de hoy, parecía más simple al inicio, no ha sido nada grave pero sí es verdad que parecía más fácil (bastante diría yo) pero bien a final la perseverancia se supo imponer, no hace falta ser un crack en esto sino tener paciencia con el objetivo nada más, el resto llega sólo ;-)

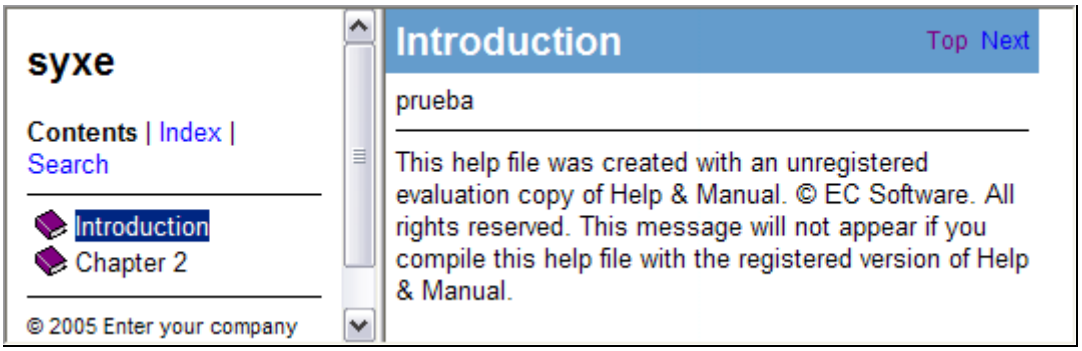
Un saludo y un abrazo a todo CrackSLatinoS por su puesto SIN EXCEPCIÓN!! Este tute lo dedico en especial a Red@Hawk por ser quien envió el programa a la lista..

Para cualquier cosa no dudéis en escribirme a "syxe00@yahoo.es"

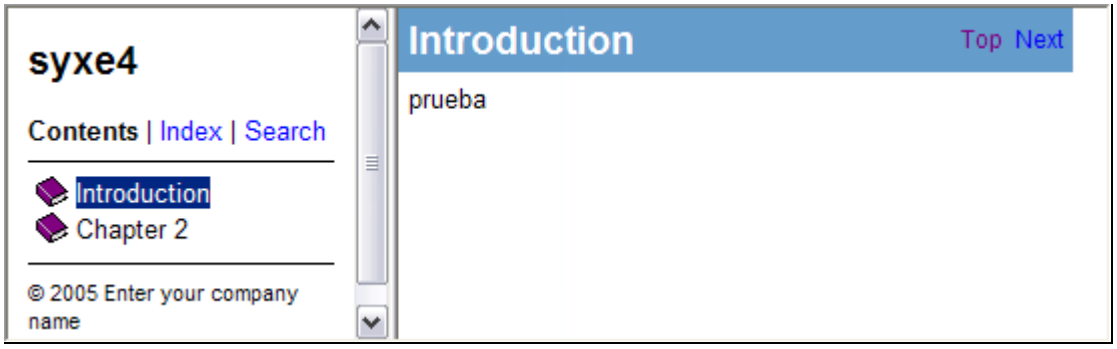
SyXe'05

ANEXO: Siguiendo las indicaciones de Red y MORALES me dio por compilar con una de las opciones que da el progama y mirad el resultado ;-)

original:



nuestro crack:



Está claro que compila sin limitaciones y sin insertar el Watermark, pienso que está bien parcheado ya que le pillamos los dos flags [16C4CF0] y [16C4CD1] que controlan todo el proceso Trial, jeje..

En fin si alguien encontrara alguna limitación o bug del crack le ruego me lo haga saber para mirarlo, nunca se sab con estas protecciones ;-)

Ahora sí me despido hasta la próxima muuuucho más tranquilo!

SyXe'05